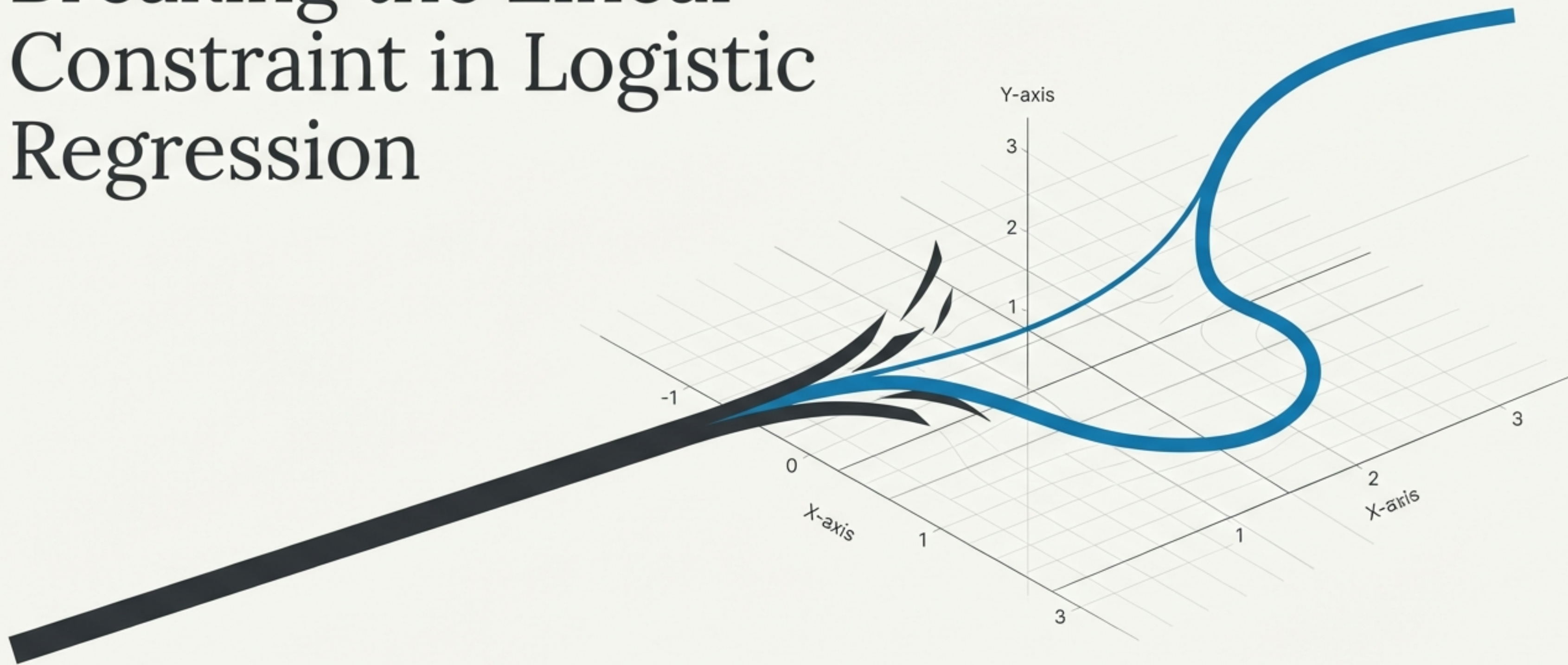


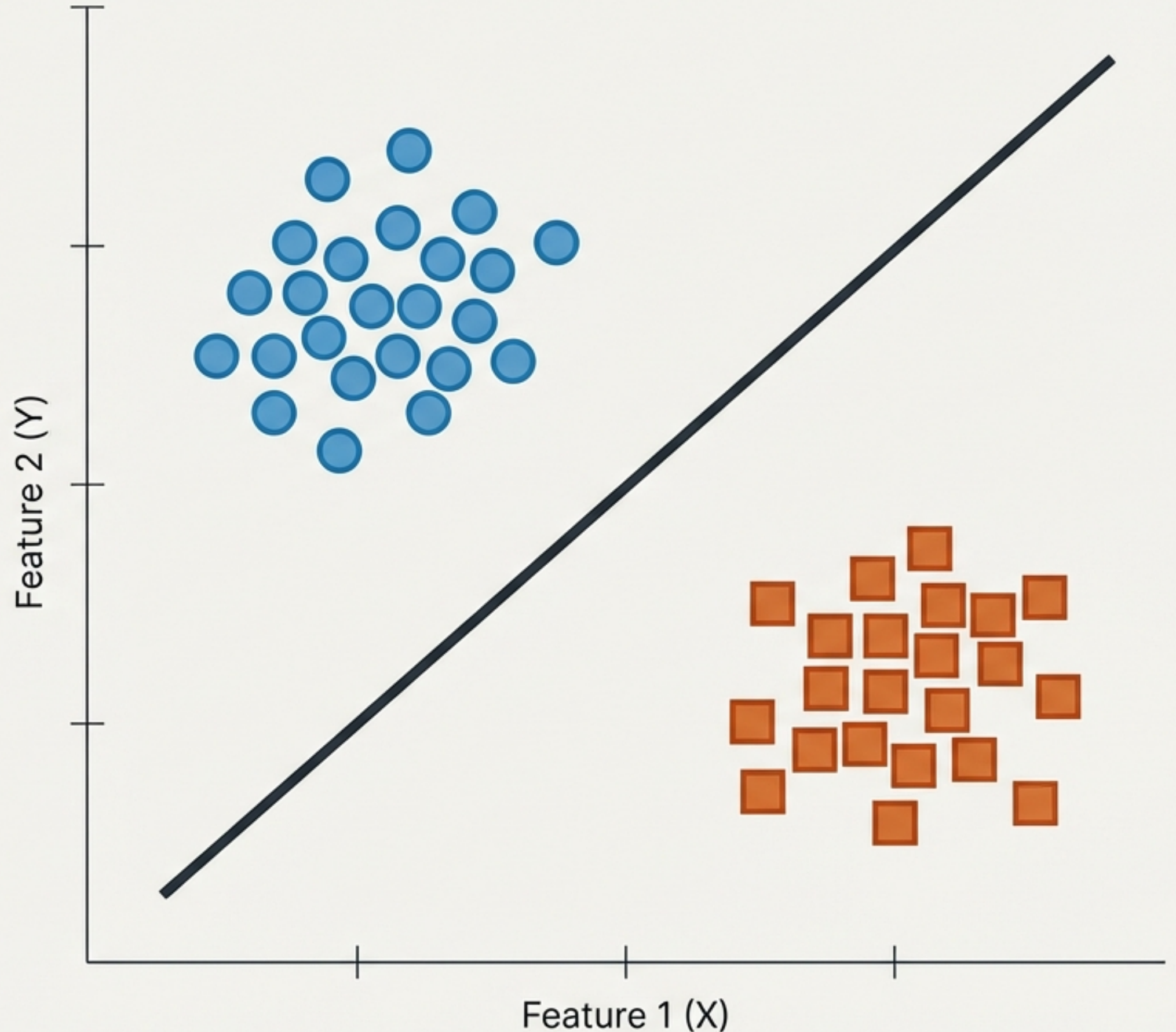
Breaking the Linear Constraint in Logistic Regression



An explainer on Polynomial Features

Vanilla logistic regression assumes strict linear behaviour

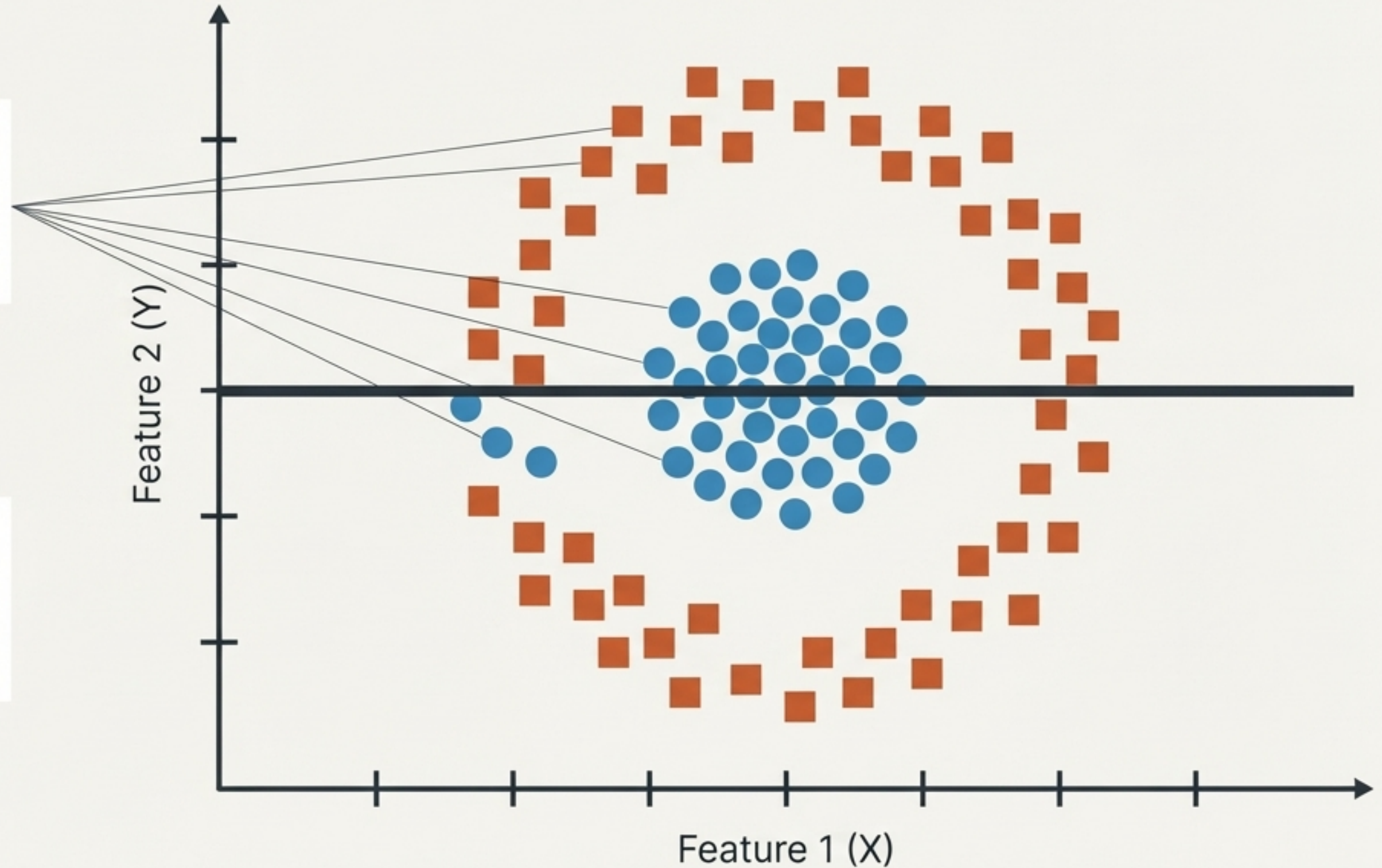
By design, standard logistic regression can only draw straight lines to separate data classes. It is fundamentally a linear classifier.



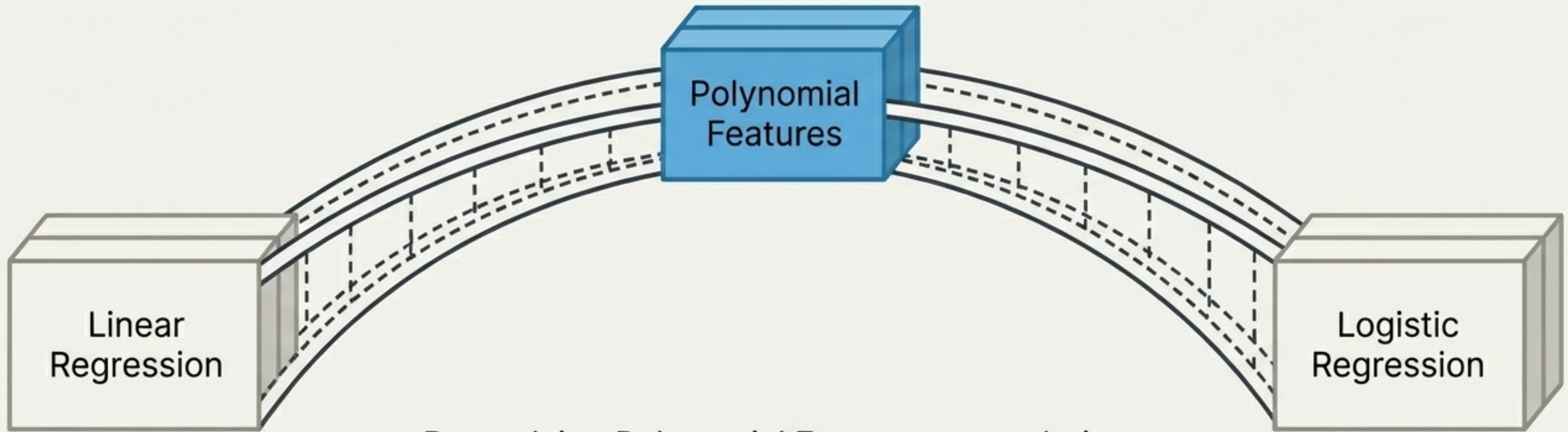
A straight line fails on non-linear data distributions

High Misclassification:
Both regions contain a
mix of classes.

Underfitting: The model is
too simple to capture the
underlying structure.



We can borrow a mathematical trick from linear models



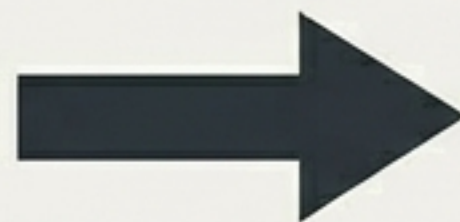
By applying Polynomial Features—a technique commonly used in linear regression—we can introduce flexibility into our logistic regression model.

Transforming the original inputs into a higher-dimensional space

We synthetically generate new input columns by calculating the powers and combinations of our original features.

X1	X2
[0.5	0.7]
[-1.2	0.3]
[2.1	-0.9]

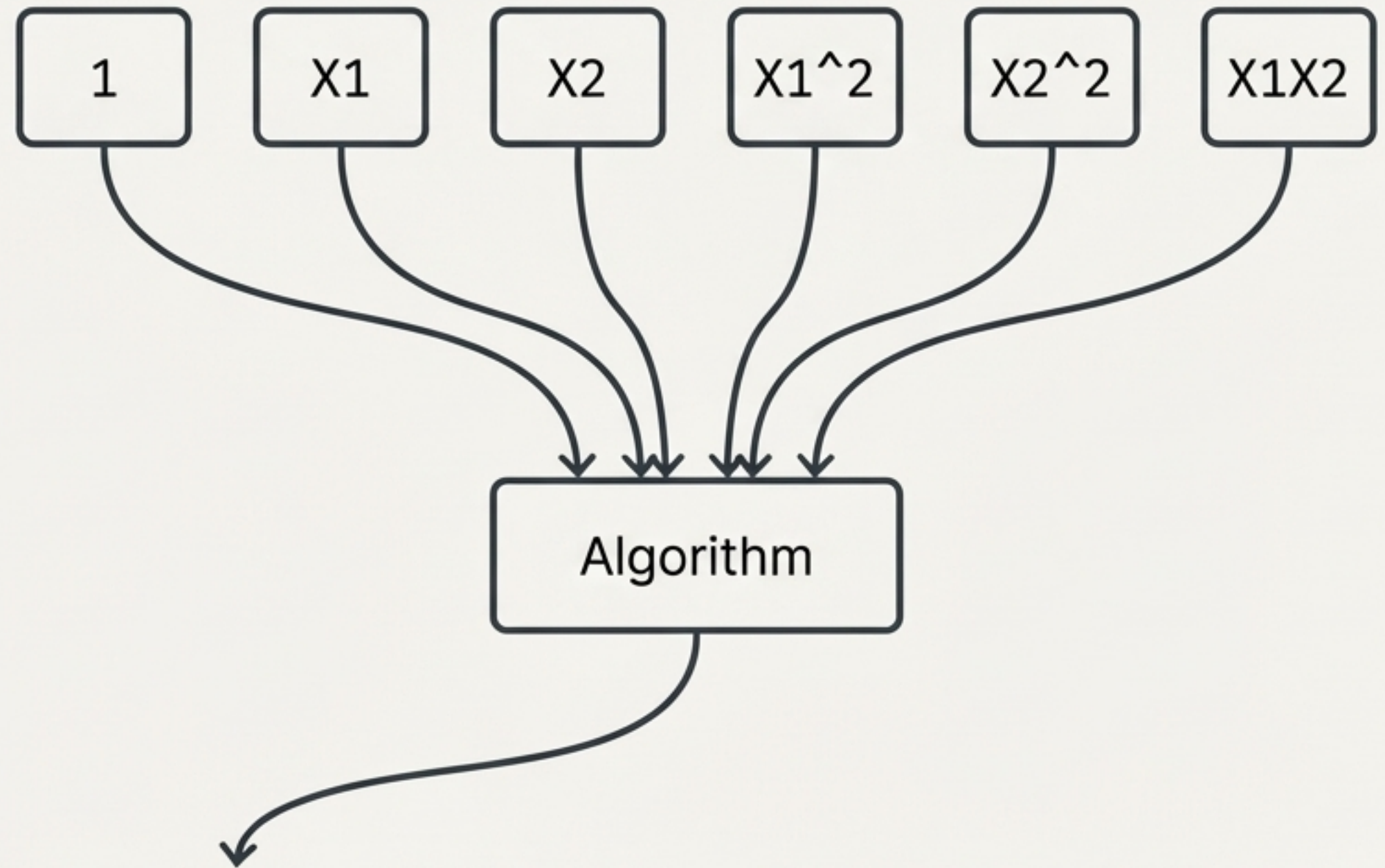
Apply Polynomial
Features
(Degree = 2)



1	X1	X2	$X1^2$	$X2^2$	$X1X2$
1	0.5	0.7	0.25	0.49	0.35
1	-1.2	0.3	1.44	0.09	-0.36
1	2.1	-0.9	4.41	0.81	-1.89

The model assigns unique weights to the newly created columns

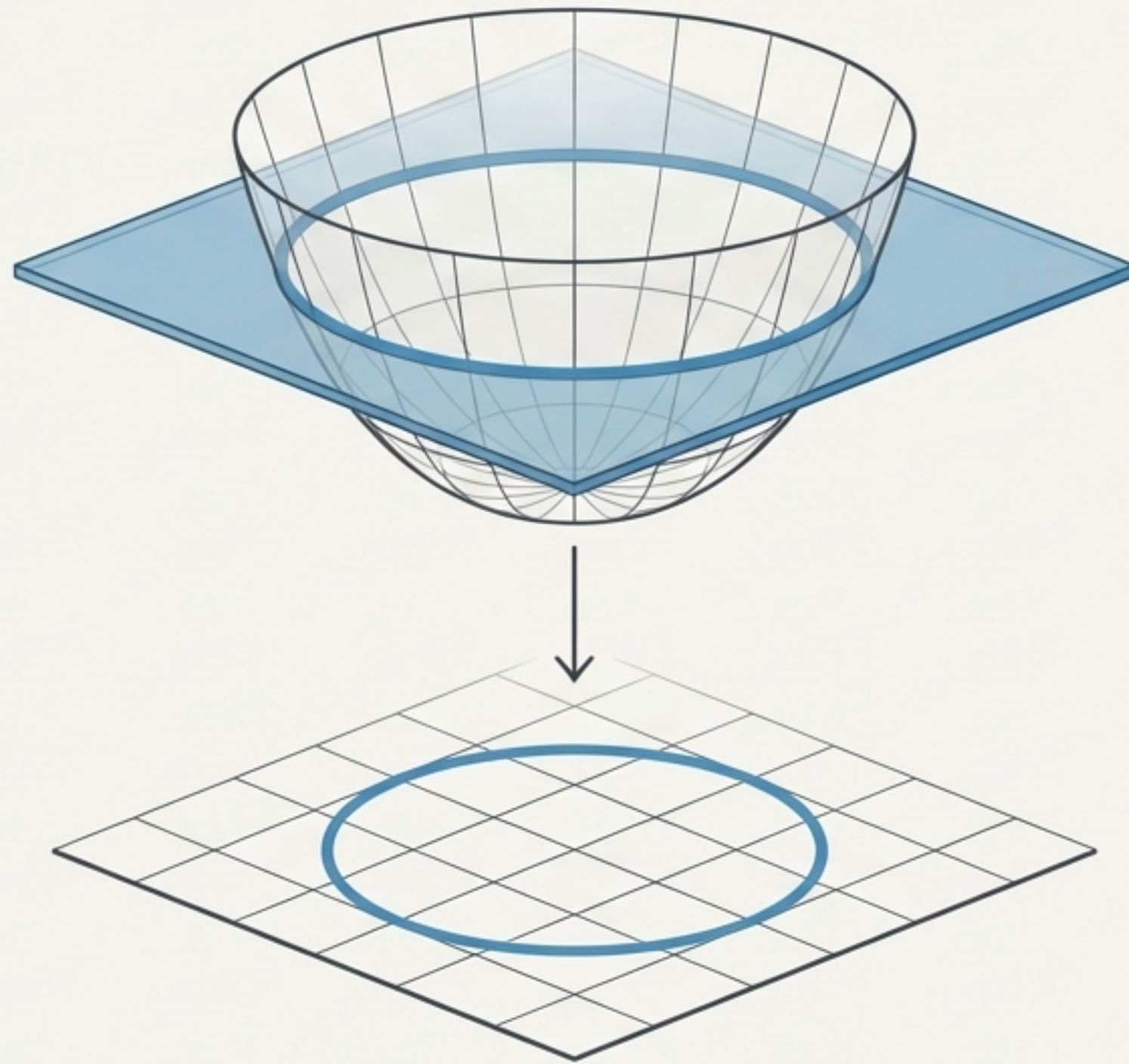
The logistic regression algorithm treats these new synthetic features exactly like normal data, optimising a unique coefficient (W) for each one.



$$Z = W_0 + W_1(X_1) + W_2(X_2) + W_3(X_2)$$

$$Z = W_0 + W_1(X_1) + W_2(X_2) + W_3(X_1^2) + W_4(X_2^2) + W_5(X_1X_2)$$

A linear boundary in higher dimensions projects as a curve in 2D

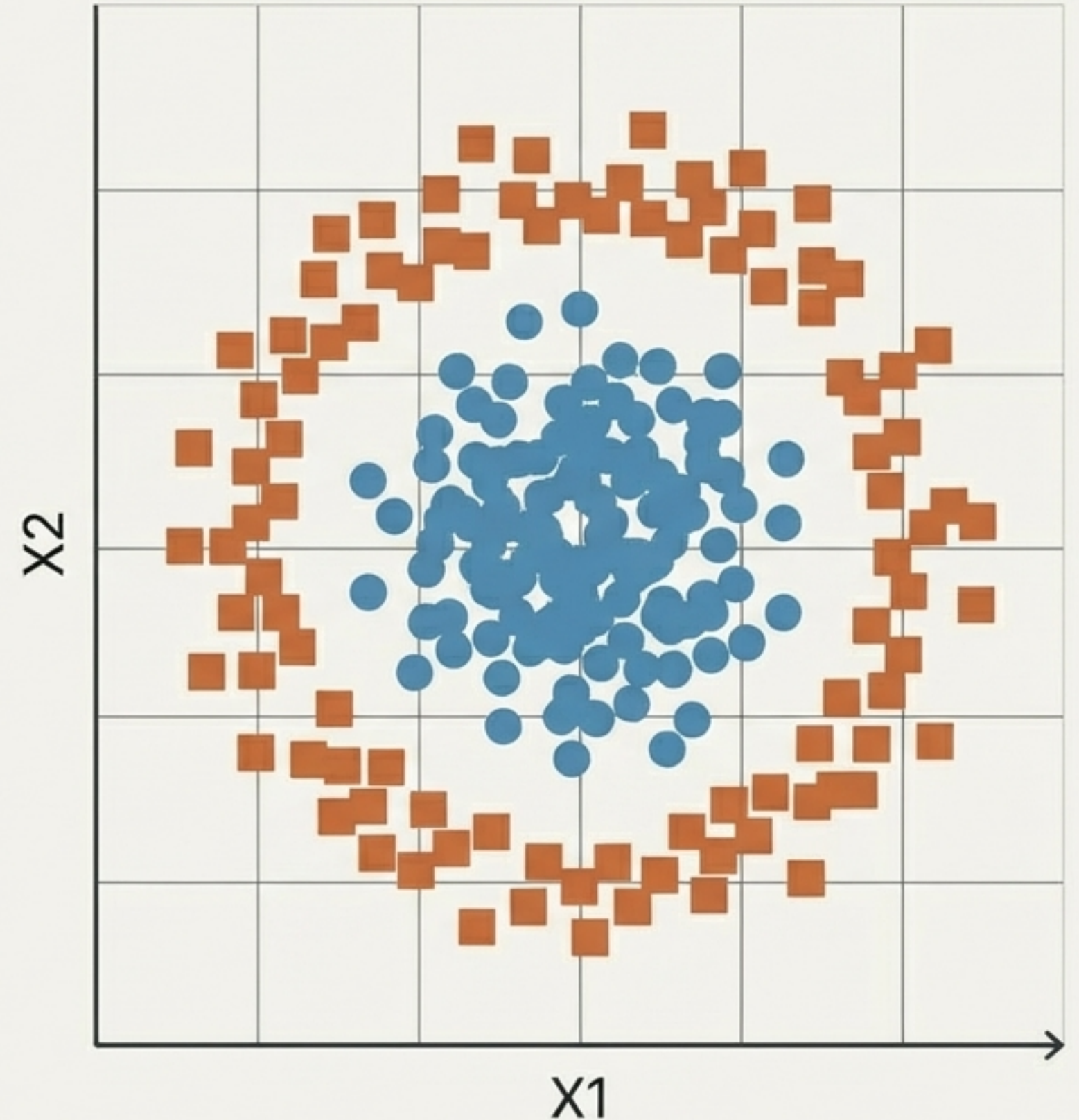


The maths remains strictly linear. The algorithm is still drawing a flat plane, but because we expanded the feature space, projecting it back down to our original 2D view reveals a curved decision boundary.

Testing polynomial degrees against a non-linear dataset

We will scale the degree hyperparameter and observe how the decision boundary evolves.

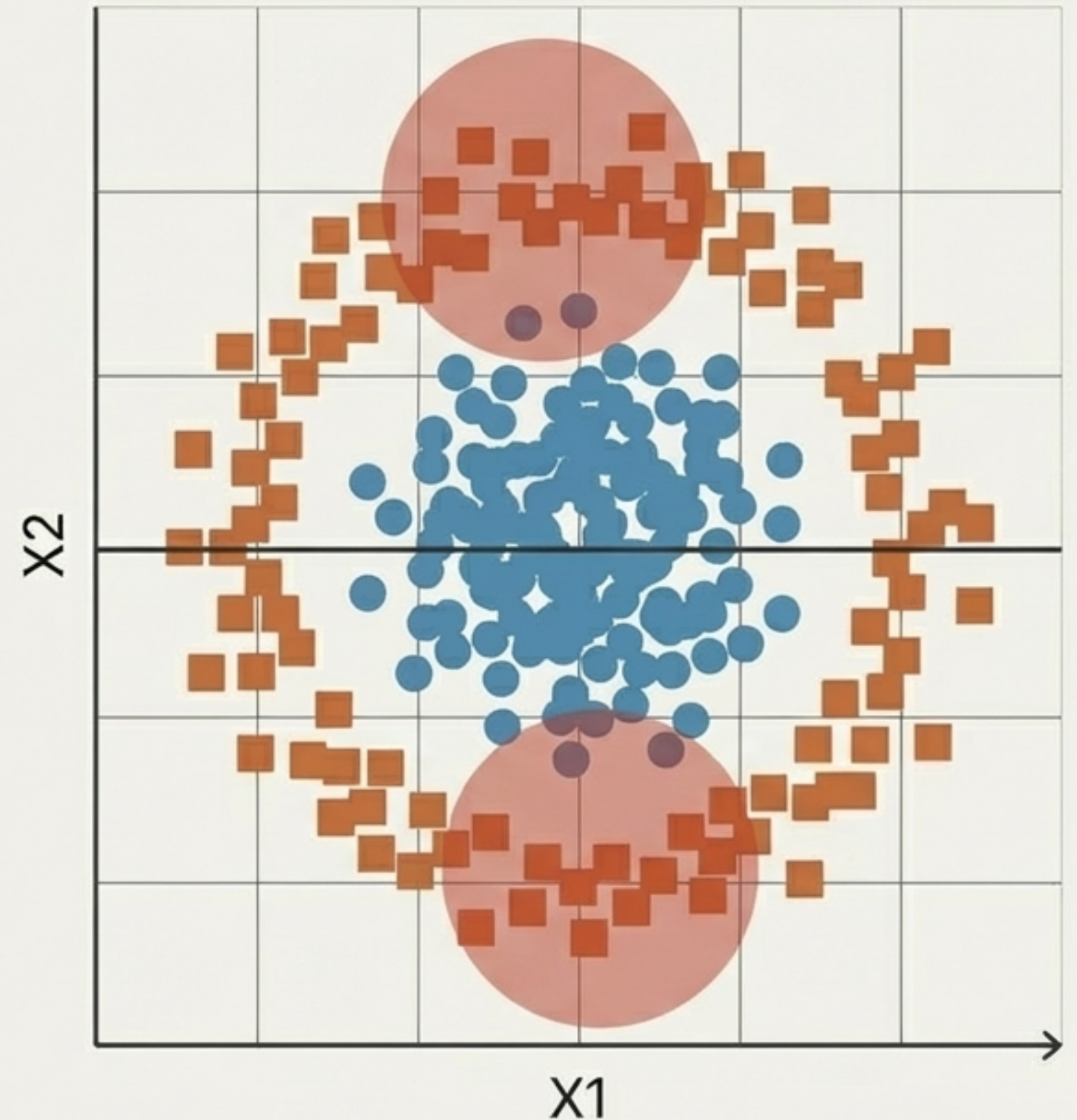
```
poly = PolynomialFeatures(degree=X)  
X_transformed = poly.fit_transform(X)  
model = LogisticRegression().fit(X_transformed, y)
```



Degree 1 forces a rigid line
and severely underfits

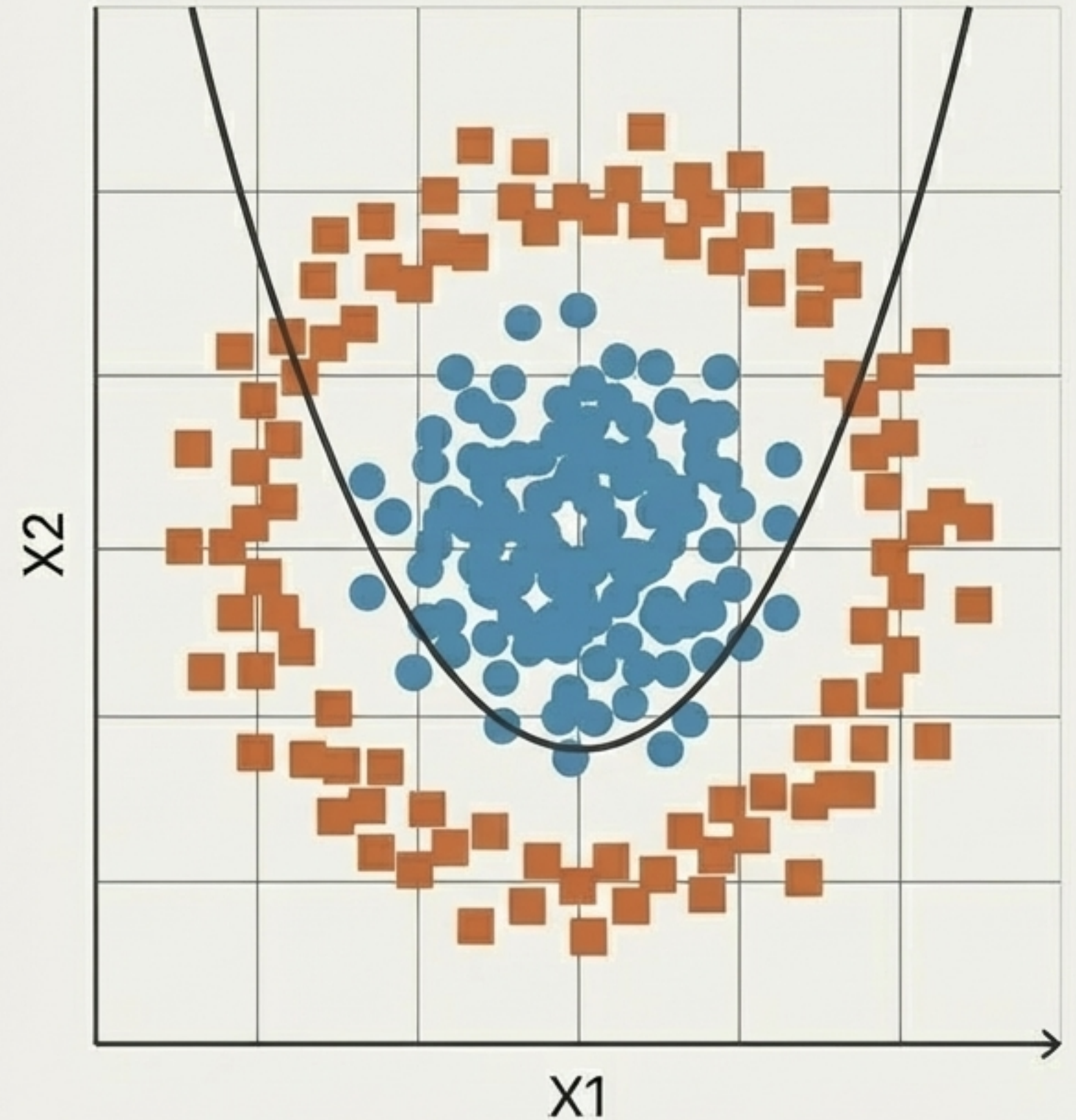
Accuracy: ~73.5%

Status: Baseline (No polynomial
features applied).



Degree 2 introduces curvature but misses the structural pattern

The boundary bends, but misclassifications persist. No significant jump in accuracy is achieved yet.

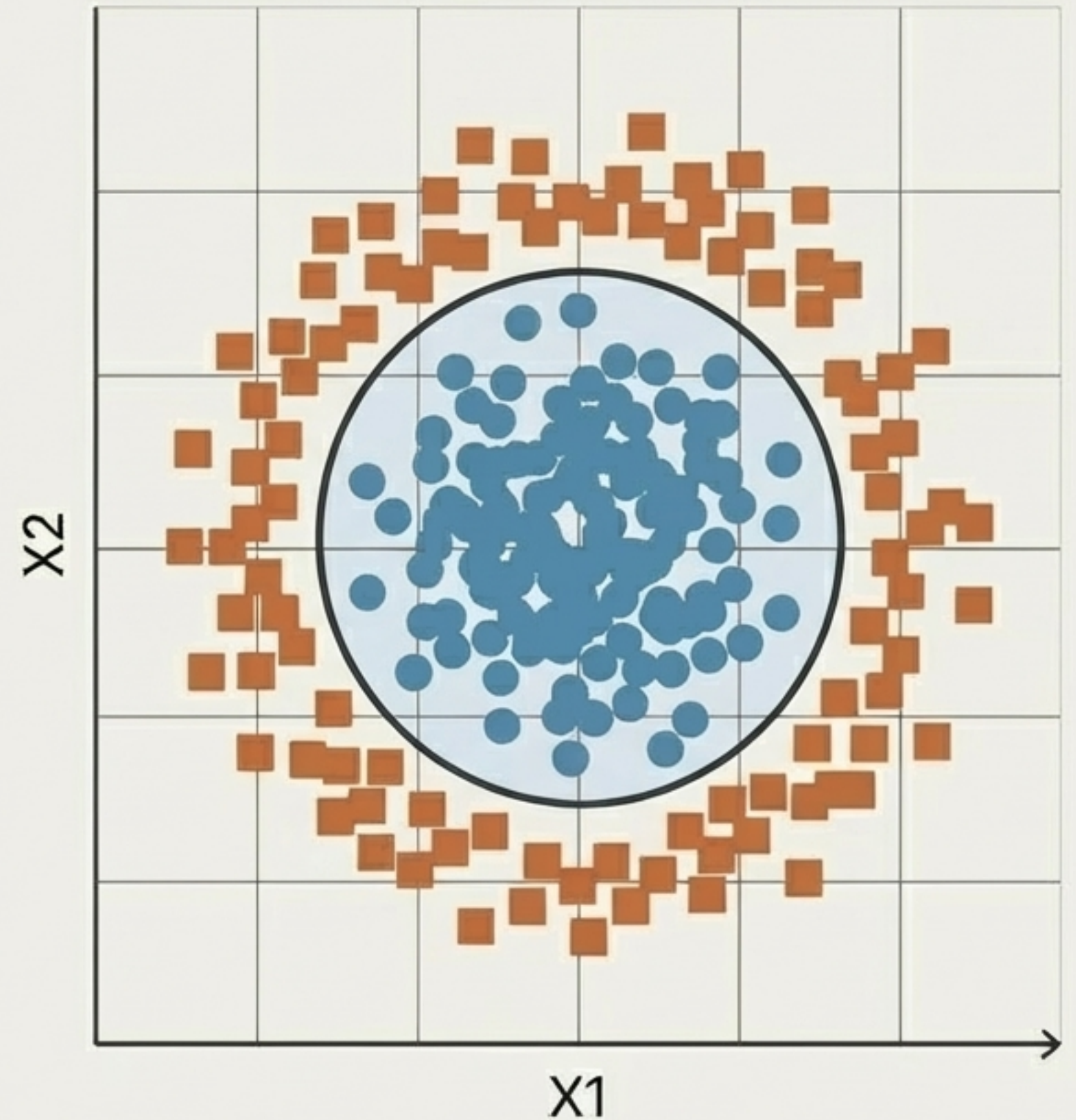


Degree 3 dynamically maps to the underlying data structure

Accuracy: ~90%

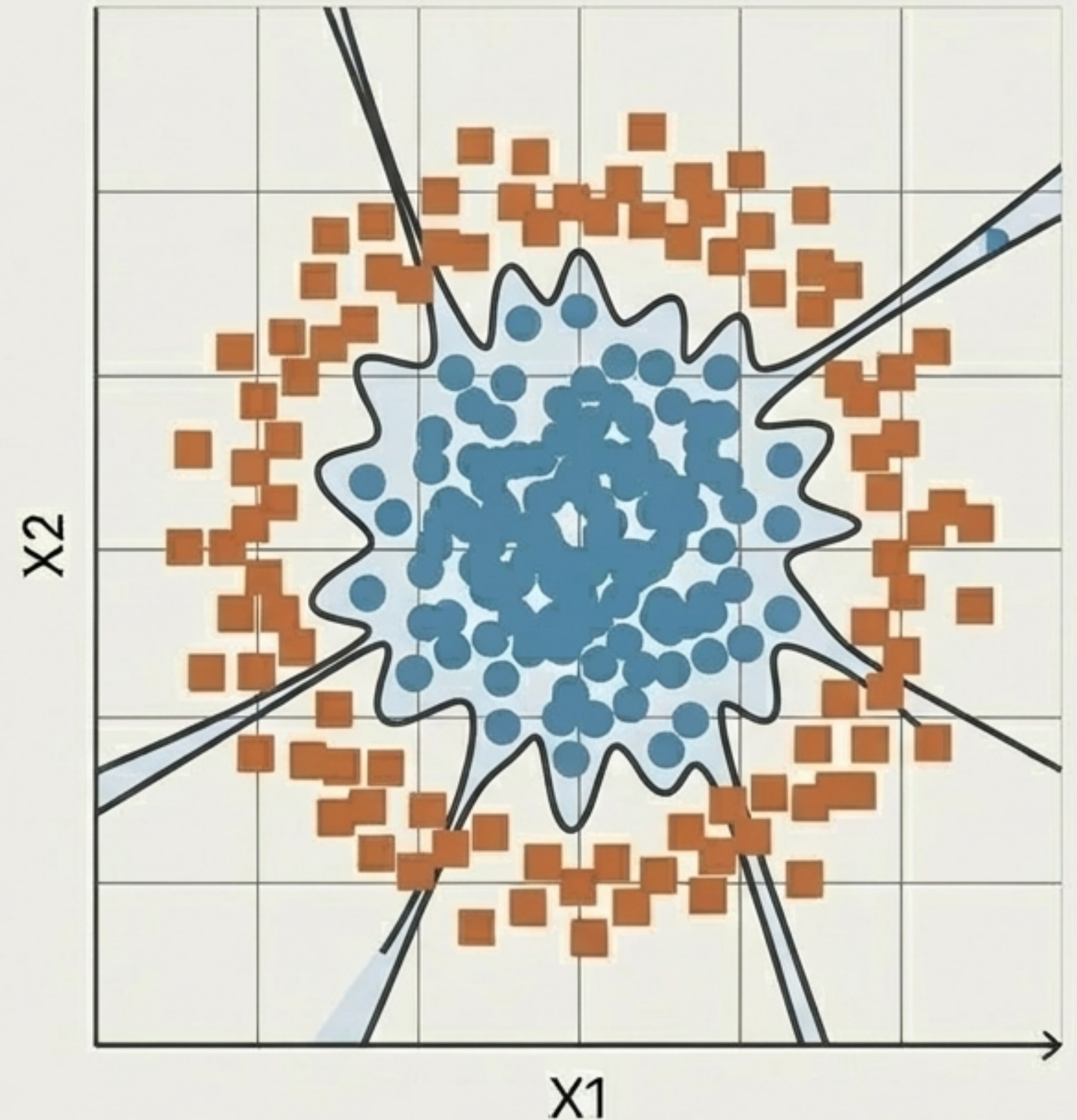
Status: Optimal Fit. The flexibility matches the data's complexity.

(Note: Degree 5 also shows slight improvements, but returns diminish).

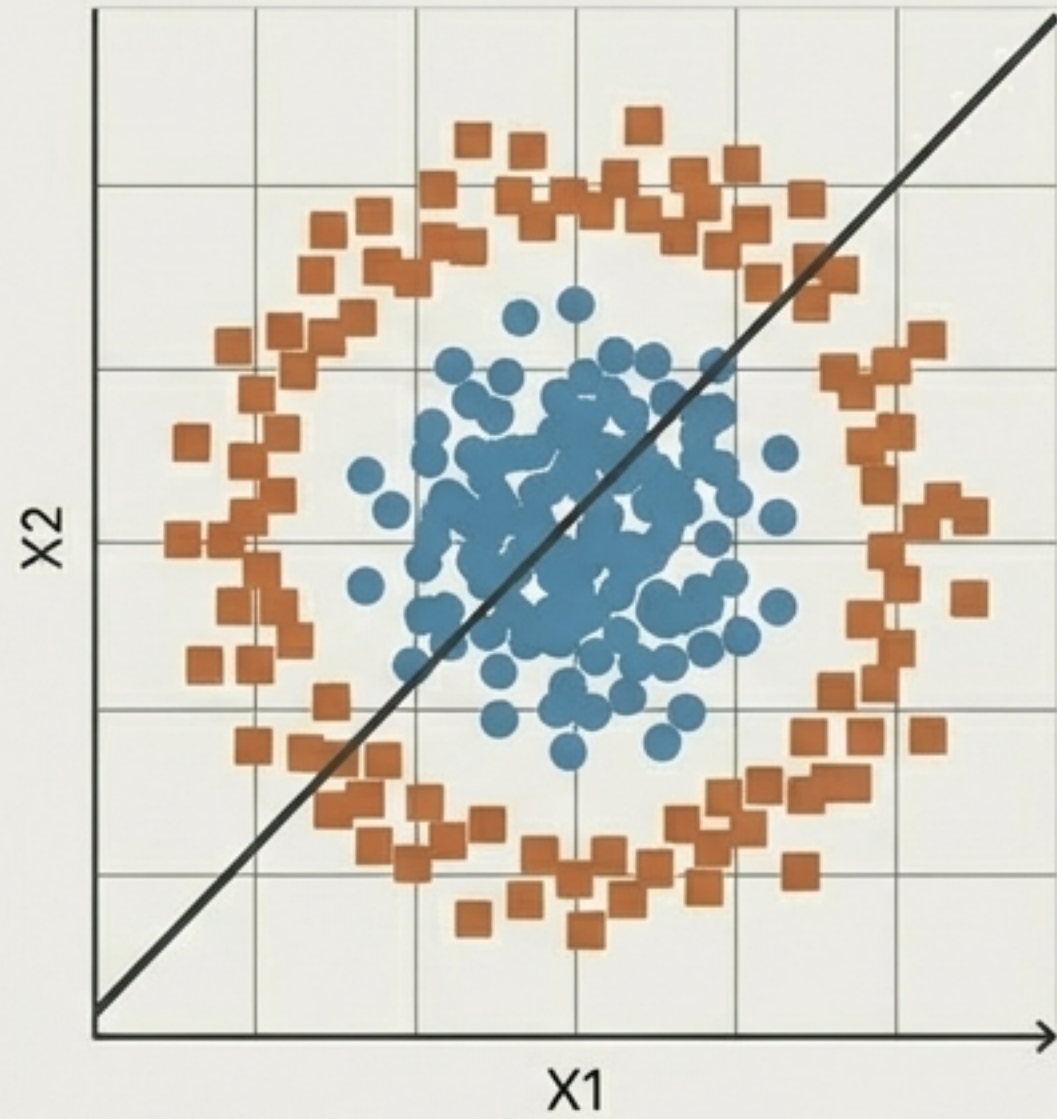


Pushing past Degree 7 causes severe model overfitting

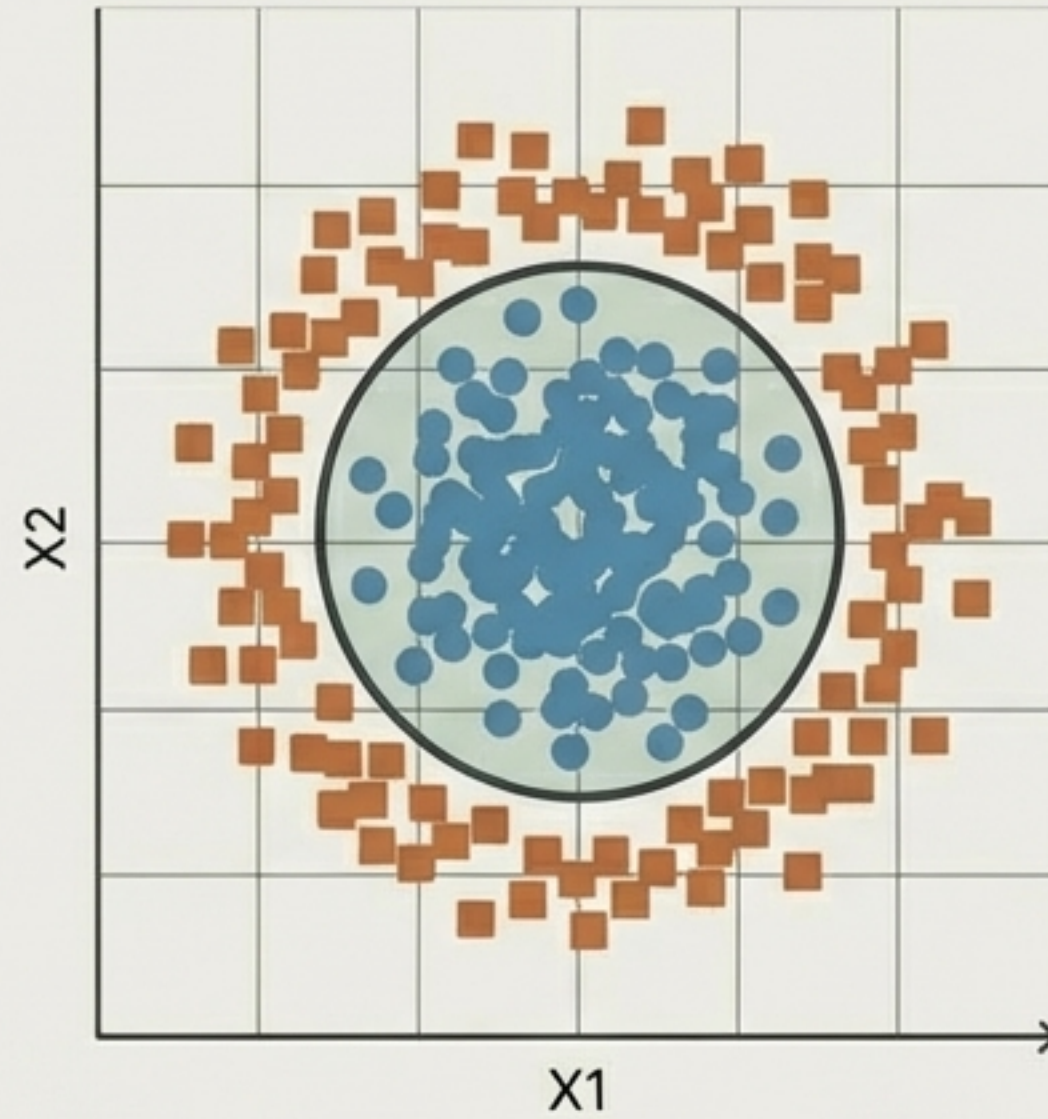
As we scale the degree higher (7+), the model gains too much flexibility. It stops learning the signal and starts memorising the noise.



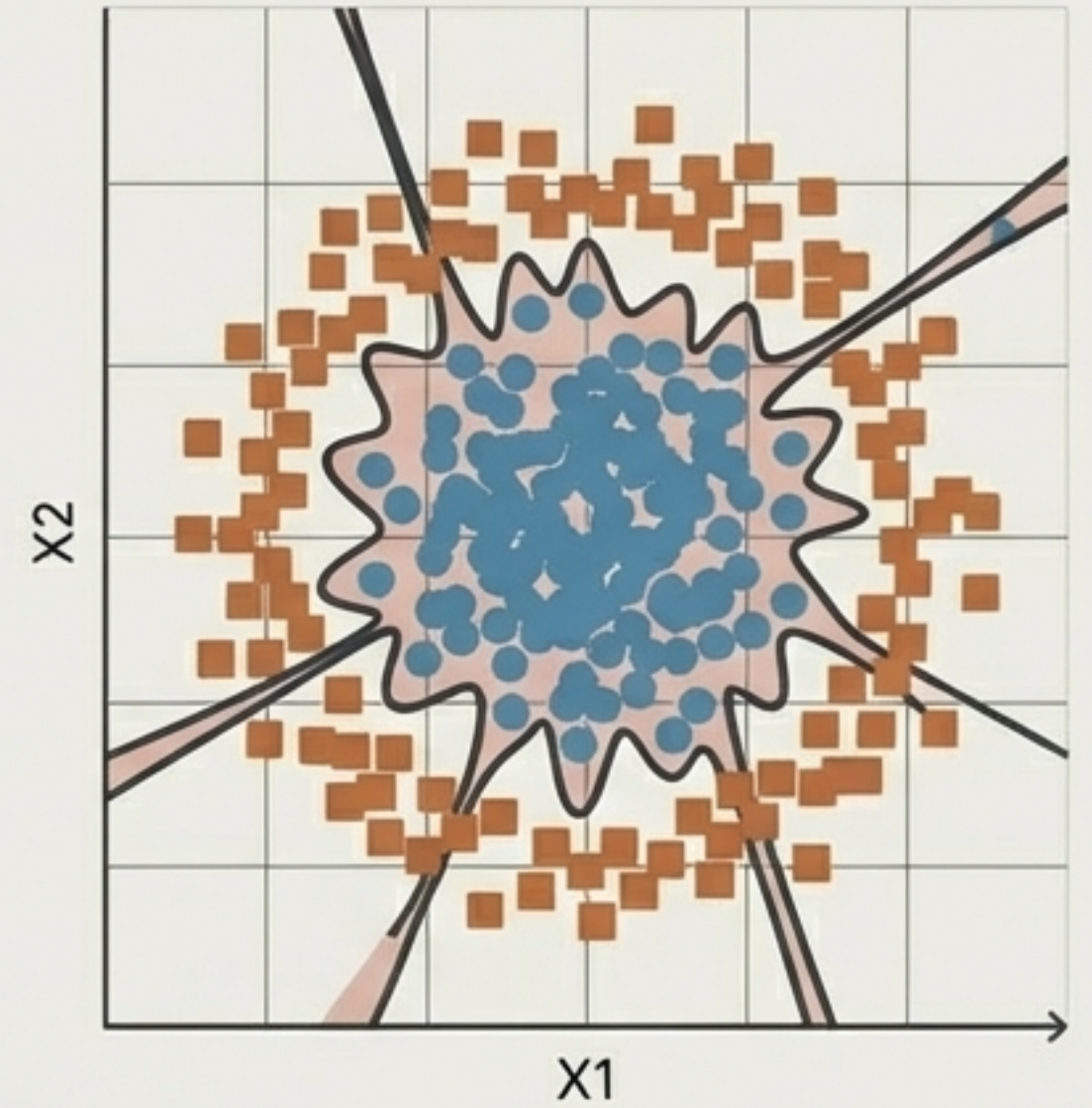
The Goldilocks zone balances accuracy and generalisation



Too Cold (Underfitting)
Degree 1



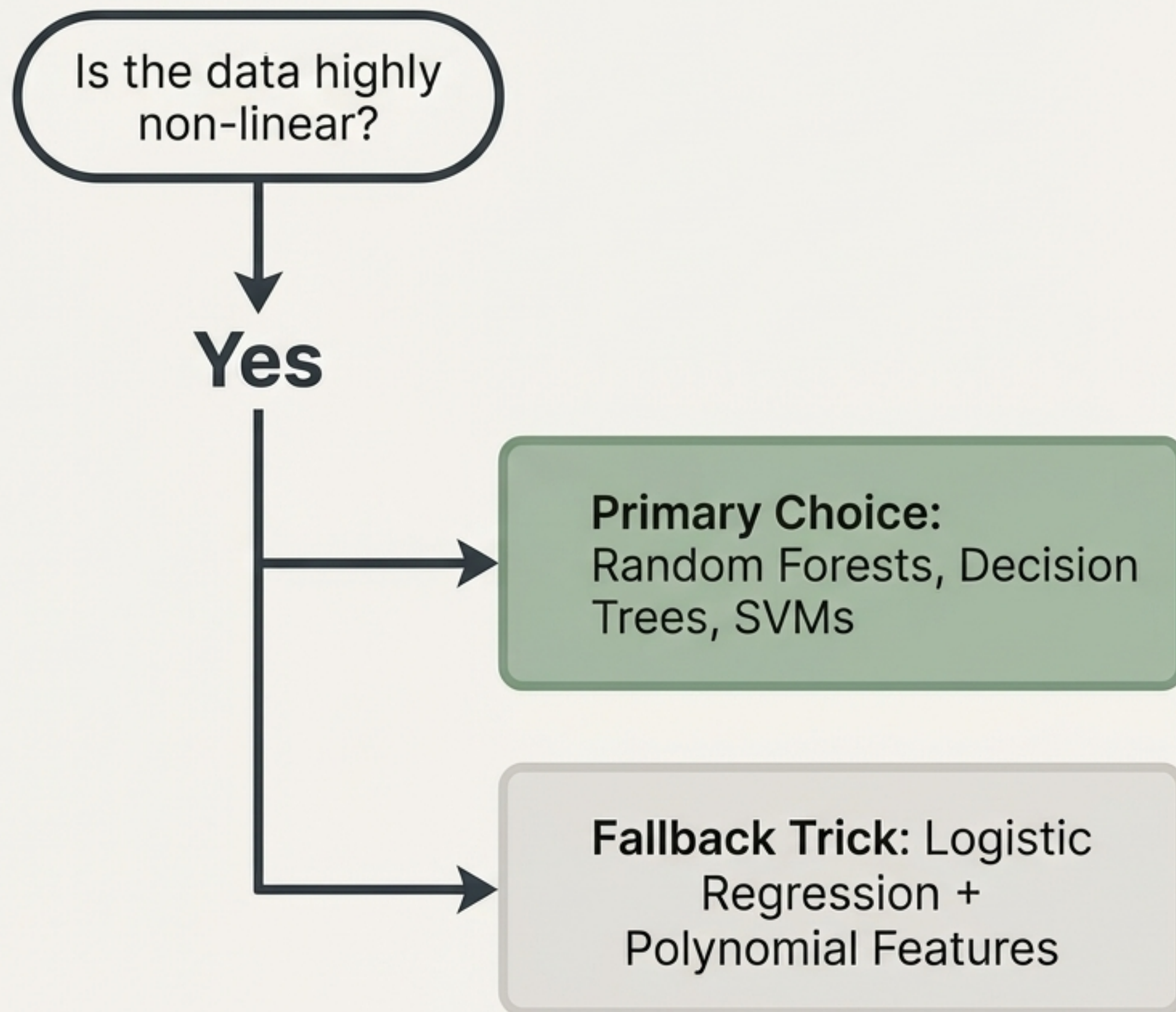
Just Right (Optimal)
Degree 3



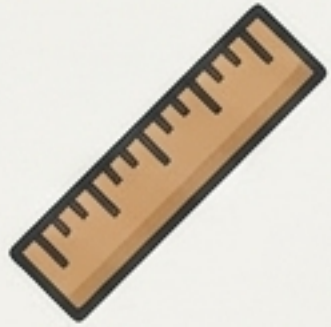
Too Hot (Overfitting)
Degree 7

Pragmatism over theory in real-world environments

While polynomial features mathematically work, tree-based models and SVMs natively handle complex non-linear data more efficiently in production environments.



Essential principles for the practitioner's toolkit



1. Vanilla logistic regression is constrained entirely to linear boundaries.



2. Polynomial feature transformation converts lower-dimensional constraints into higher-dimensional curves.



3. Always benchmark this technique against native non-linear algorithms like Random Forests to ensure optimal performance.